Axios

一、axios 是什么

二、axios 特点

三、axios 常用语法

四、axios 使用

- 0、使用方法
- 1、axios的基本使用：axios(config)
- 2、axiosCRUD方法：get、post、delete、put
- 3、响应结果介绍
- 4、axios 配置对象 config
- 5、默认配置：axios.defaults.xxx
- 6、axios 创建实例对象发送请求：axios.create([config])
- 7、拦截器：axios.interceptors
- 8、取消请求：axios.CancelToken()

五、

六、

七、

- 1、
- 2、
- 3、
- 4、
- 5、
- 6、
- 7、
- 8、
- 9、
  - ①、
  - ②、
  - ③、

④、
⑤、
⑥、
⑦、
⑧、
⑨、
⑩、
Ⅰ、
Ⅱ、
Ⅲ、
Ⅳ、
Ⅴ、
Ⅵ、
Ⅶ、
Ⅷ、
Ⅸ、
Ⅹ、

> https://www.bilibili.com/video/BV1wr4y1K7tq/?p=11&spm_id_from=pageDriver&vd_source=b55e15966ca689b32671d4aa387cab01
>
> 📎尚硅谷_axios从入门到源码分析.pdf
>
> 代码：📎3.jpg

一、axios 是什么

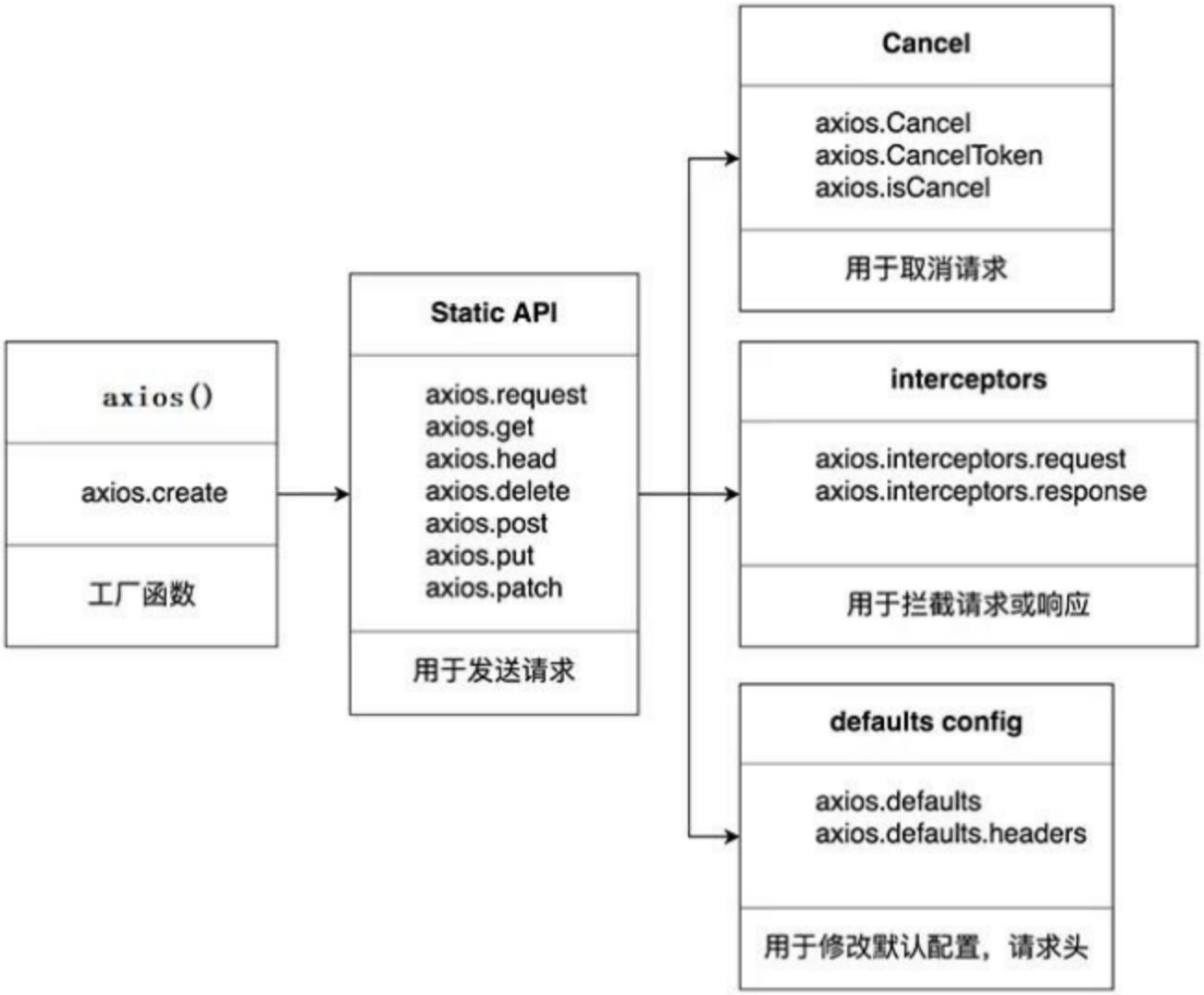
1. 前端最流行的 ajax 请求库
2. react/vue 官方都推荐使用 axios 发 ajax 请求
3. 文档：https://github.com/axios/axios

二、axios 特点

1. 基于 xhr + promise 的异步 ajax 请求库
2. 浏览器端/node 端都可以使用
3. 支持请求 / 响应拦截器
4. 支持请求取消
5. 请求/响应数据转换
6. 批量发送多个请求

三、axios 常用语法

语法（方括号内为可选参数）	描述
`axios(config)`	通用/最本质的发任意类型请求的方式
`axios.request(config)`	等同于 `axios(config)`
`axios(url[, config])`	可以只指定 url 发 get 请求
`axios.get(url[, config])`	发 get 请求
`axios.delete(url[, config])`	发 delete 请求
`axios.post(url[, data, config])`	发 post 请求
`axios.put(url[, data, config])`	发 put 请求
`axios.defaults.xxx`	请求的默认全局配置
`axios.interceptors.request.use()`	添加请求拦截器
`axios.interceptors.response.use()`	添加响应拦截器
`axios.create([config])`	创建一个新的 axios(它没有下面的功能)
`axios.Cancel()`	用于创建取消请求的错误对象
`axios.CancelToken()`	用于创建取消请求的 token 对象
`axios.isCancel()`	是否是一个取消请求的错误
`axios.all(promises)`	用于批量执行多个异步请求
`axios.spread()`	用来指定接收所有成功数据的回调函数的方法



四、axios 使用

0、使用方法

1. npm： `npm install axios`
2. yarn： `yarn add axios`
3. bower： `bower install axios`
4. Using jsDelivr CDN： `<script src="https: //cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>`
5. Using unpkg CDN： `<script src="https: //unpkg.com/axios/dist/axios.min.js"></script>`

> Boot CDN：https://www.bootcdn.cn/

- Spring Boot 项目设置允许跨域：在 Controller 类或者方法上添加注解：`@CrossOrigin`

Java | 复制代码

```java
package com.yuehai.controller;

import org.springframework.web.bind.annotation.*;

import java.util.Map;

/**
 * @author 10222148
 * @create 2022/11/23 13:10
 */

@CrossOrigin
@RestController
public class AxiosController {

    @GetMapping("/hello")
    public String getHello() {
        return "Hello Spring Boot 2 GET";
    }

    @PostMapping("/hello")
    public String postHello(@RequestBody Map<String, Object> map) {
        return "Hello Spring Boot 2 POST; 姓名: " + map.get("name") + ", 年龄: " + map.get("age");
    }

    @DeleteMapping("/hello")
    public String deleteHello() {
        return "Hello Spring Boot 2 DELETE";
    }

    @PutMapping("/hello")
    public String putHello() {
        return "Hello Spring Boot 2 PUT";
    }
}

```

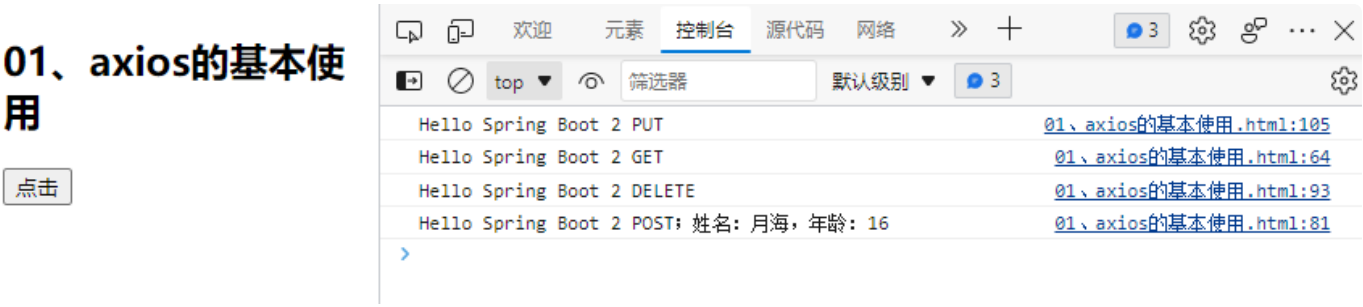
1、axios的基本使用：`axios(config)`

1. `axios(config)`：通用/最本质的发任意类型请求的方式
2. `axios.request(config)`：等同于 `axios(config)`

HTML | 复制代码

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- 引入 axios -->
    <script src="https://cdn.bootcdn.net/ajax/libs/axios/1.1.3/axios.js">
</script>
    <title>Document</title>
</head>
<body>
    <h2>01、axios的基本使用</h2>
    <button id="btn">点击</button>
</body>

<script>

    // 定义使用 axios 发送 get 请求的函数
    function getAxios() {
        axios({
            // 请求类型
            method: 'GET',
            // 请求地址
            url: 'http://localhost:8080/hello'
        }).then(response =>{
            console.log(response.data);
        })
    }

    // 定义使用 axios 发送 post 请求的函数
    function postAxios(name, age) {
        axios({
            // 请求类型
            method: 'post',
            // 请求地址
            url: 'http://localhost:8080/hello',
            // 请求体
            data: {
                name: name,
                age: age
            }
        }).then(response =>{
            console.log(response.data);
        })
```

```
    }

    // 定义使用 axios 发送 delete 请求的函数
    function deleteAxios() {
        axios({
            // 请求类型
            method: 'delete',
            // 请求地址
            url: 'http://localhost:8080/hello'
        }).then(response =>{
            console.log(response.data);
        })
    }

    // 定义使用 axios 发送 put 请求的函数
    function putAxios() {
        axios({
            // 请求类型
            method: 'put',
            // 请求地址
            url: 'http://localhost:8080/hello'
        }).then(response =>{
            console.log(response.data);
        })
    }

    // 获取元素对象
    const btn = document.querySelector("#btn")
    // 绑定单击事件
    btn.addEventListener("click",function(){

        // 调用 getAxios 方法
        getAxios();
        // 调用 postAxios 方法
        postAxios('月海',16);
        // 调用 deleteAxios 方法
        deleteAxios();
        // 调用 putAxios 方法
        putAxios();

    })

</script>

</html>
```



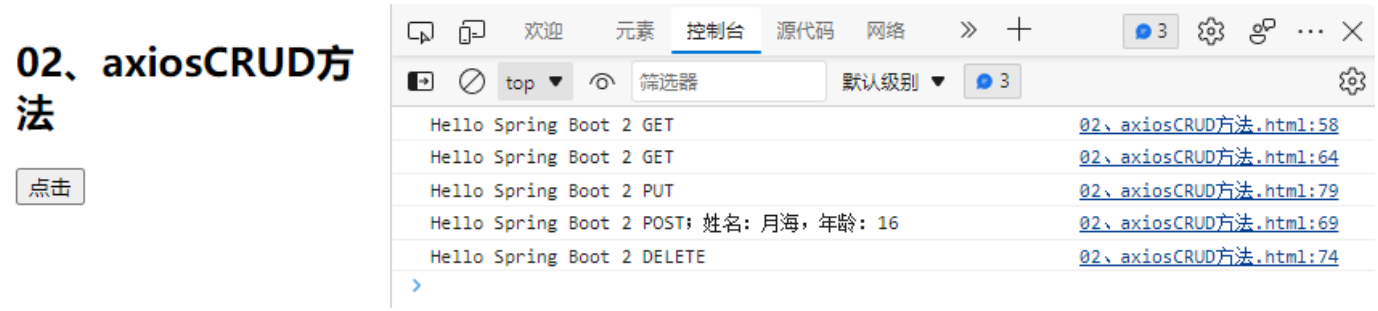
2、axiosCRUD方法：`get、post、delete、put`

1. `axios(url[, config])`：可以只指定 url 发 get 请求
2. `axios.get(url[, config])`：发 get 请求
3. `axios.delete(url[, config])`：发 delete 请求
4. `axios.post(url[, data, config])`：发 post 请求
5. `axios.put(url[, data, config])`：发 put 请求

HTML | 复制代码

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- 引入 axios -->
    <script src="https://cdn.bootcdn.net/ajax/libs/axios/1.1.3/axios.js">
</script>
    <title>Document</title>
</head>
<body>
    <h2>02、axiosCRUD方法</h2>
    <button id="btn">点击</button>
</body>

<script>

    // axios(url[, config]) 可以只指定 url 发 get 请求
    function getAxios1() {
        axios('http://localhost:8080/hello').then(data =>{ console.log(dat
a.data) })
    }


    // axios.get(url[, config]) 发 get 请求
    function getAxios2() {
        axios.get('http://localhost:8080/hello').then(data =>{ console.log
(data.data) })
    }

    // axios.post(url[, data, config])  发 post 请求
    function postAxios(name, age) {
        axios.post('http://localhost:8080/hello', { name: name, age: age }
).then(data =>{ console.log(data.data) })
    }

    // axios.delete(url[, config])  发 delete 请求
    function deleteAxios() {
        axios.delete('http://localhost:8080/hello').then(data =>{ console.
log(data.data) })
    }

    // axios.put(url[, data, config]) 发 put 请求
    function putAxios() {
```

```
41          axios.put('http://localhost:8080/hello').then(data =>{ console.log
(data.data) })
42      }
43
44
45      // 获取元素对象
46      const btn = document.querySelector("#btn")
47      // 绑定单击事件
48      btn.addEventListener("click",function(){
49
50          // 调用 getAxios1 方法
51          getAxios1();
52          // 调用 getAxios2 方法
53          getAxios2();
54          // 调用 postAxios 方法
55          postAxios('月海',16);
56          // 调用 deleteAxios 方法
57          deleteAxios();
58          // 调用 putAxios 方法
59          putAxios();
60
61      })
62
63  </script>
64
65  </html>
```



3、响应结果介绍

4、axios 配置对象 `config`

参数	描述
`url`	请求地址
`method`	请求类型
`baseURL`	设置 `url` 的基础结构；最终的请求地址会将 `url` 和 `baseURL` 结合后发送
`transformRequest`	对请求的数据进行处理，将处理后的结果发送到服务器
`transformResponse`	对接收的数据进行处理，处理完之后我们再使用回调方法对其进行操作
`headers`	配置请求头信息
`params`	设置 `url` 参数
`paramsSerializer`	对请求的参数进行序列号，变为一个字符串
`data`	设置请求体；可以为对象或者字符串
`timeout`	超时时间，单位毫秒；超时这个请求将被取消
`withCredentials`	设置跨域请求时 cookie 的携带；false 为不携带
`adapter`	对请求的识别器进行设置；发送 AJAX 或者 HTTP 请求
`auth`	请求的基础验证，设置用户名密码等
`responseType`	设置响应体格式；默认 json
`responseEncoding`	设置响应体编码；默认 utf-8
`xsrfCookieName`	跨站请求标识；对 cookie 的名字进行设置
`xsrfHeaderName`	跨站请求标识；对请求头进行设置
`onUploadProgress`	上传时的回调函数
`onDownloadProgress`	下载时的回调函数
`maxContentLength`	设置响应体的最大容量；单位为字节
`maxBodyLength`	设置请求体的最大容量；单位为字节
`validateStatus`	设置什么情况下请求才是成功的
`maxRedirects`	最大跳转次数

`beforeRedirect`	
`socketPath`	向 docker 的守护进程发送请求，做数据转发
`httpAgent`	对客户端的信息进行设置
`httpsAgent`	同上
`proxy`	设置代理（一般用在服务端 nodejs）
`cancelToken`	取消 ajax 请求
`signal`	
`decompress`	对响应结果进行解压（服务端 nodejs）
`insecureHTTPParser`	
`transitional`	
`silentJSONParsing`	
`forcedJSONParsing`	
`clarifyTimeoutError`	
`env`	
`formSerializer`	
`maxRate`	

5、默认配置：`axios.defaults.xxx`

HTML | 复制代码

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- 引入 axios -->
    <script src="https://cdn.bootcdn.net/ajax/libs/axios/1.1.3/axios.js">
</script>
    <title>Document</title>
</head>
<body>
    <h2>03、axios 默认配置</h2>
    <button id="btn">点击</button>
</body>

<script>

    // 进行 axios 默认配置
    // 配置默认请求类型
    axios.defaults.method = 'POST';
    // 配置基础 url
    axios.defaults.baseURL = 'http://localhost:8080';

    // 定义使用 axios 发送 post 请求的函数
    function postAxios(name, age) {
        axios({
            // 请求类型
            method: 'post',
            // 请求地址
            url: '/hello',
            // 请求体
            data: {
                name: name,
                age: age
            }
        }).then(response =>{
            console.log(response.data);
        })
    }

    // 获取元素对象
    const btn = document.querySelector("#btn")
    // 绑定单击事件
    btn.addEventListener("click",function(){
```

```
45
46          // 调用 postAxios 方法
47          postAxios('月海',16);
48
49      })
50
51  </script>
52
53  </html>
```

6、axios 创建实例对象发送请求：`axios.create([config])`

HTML | 复制代码

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- 引入 axios -->
    <script src="https://cdn.bootcdn.net/ajax/libs/axios/1.1.3/axios.js">
</script>
    <title>Document</title>
</head>
<body>
    <h2>04、axios 创建实例对象发送请求</h2>
    <button id="btn">点击</button>
</body>

<script>

    // 获取元素对象
    const btn = document.querySelector("#btn")
    // 绑定单击事件
    btn.addEventListener("click",function(){

        // 创建实例对象
        const duanzi = axios.create({
            baseURL: 'http://localhost:8080'
        })
        // 这里 duanzi 与 axios 的功能几乎时一样的
        duanzi({
            url: '/hello'
        }).then((data) => {
            console.log(data.data);
        })

        // 也可以调用方法
        duanzi.post('/hello',{ name: '言', age: 14 }).then((data) => {
            console.log(data.data);
        })


    })

</script>

</html>
```

7、拦截器：`axios.interceptors`

1. 说明：调用 axios() 并不是立即发送 ajax 请求，而是需要经历一个较长的流程
2. 流程：请求拦截器2 => 请求拦截器1 => 发 ajax 请求 => 响应拦截器1 => 响应拦截器 2 => 请求的回调
3. 注意：此流程是通过 promise 串连起来的，请求拦截器传递的是 config，响应拦截器传递的是 response

HTML 复制代码

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- 引入 axios -->
    <script src="https://cdn.bootcdn.net/ajax/libs/axios/1.1.3/axios.js">
</script>
    <title>Document</title>
</head>

<body>
    <h2>05、拦截器</h2>
    <button id="btn">点击</button>
</body>

<script>

    // 设置请求拦截器 1
    axios.interceptors.request.use(config => {
        console.log("请求拦截器 1，成功");
        // 在请求拦截器中可以对请求的配置进行设置
        config.params = {a:100, b:200};
        console.log("在请求拦截器 1 中设置参数完成");
        return config;
    }, error => {
        console.log("请求拦截器 1，失败");
        return Promise.reject(error);
    });

    // 设置请求拦截器 2
    axios.interceptors.request.use(config => {
        console.log("请求拦截器 2，成功");
        return config;
    }, error => {
        console.log("请求拦截器 2，失败");
        return Promise.reject(error);
    });

    // 设置响应拦截器 1
    axios.interceptors.response.use(response => {
        console.log("响应拦截器 1，成功");
        // 在响应拦截器中也可以对结果进行处理
```

```
45          console.log(`返回的数据为: ${response.data}`);
46          return response;
47      }, error => {
48          console.log("响应拦截器 1, 失败");
49          return Promise.reject(error);
50      });
51
52      // 设置请求拦截器 2
53      axios.interceptors.request.use(config => {
54          console.log("响应拦截器 2, 成功");
55          return config;
56      }, error => {
57          console.log("响应拦截器 2, 失败");
58          return Promise.reject(error);
59      });
60
61      // 进行 axios 默认配置
62      // 配置基础 url
63      axios.defaults.baseURL = 'http://localhost:8080';
64
65      // 定义使用 axios 发送 post 请求的函数
66      function postAxios(name, age) {
67          axios.post('/hello', { name: name, age: age })
68          .then(res => {
69              console.log(res)
70          })
71          .catch(err => {
72              console.error(err);
73          })
74      }
75
76      // 获取元素对象
77      const btn = document.querySelector("#btn")
78      // 绑定单击事件
79      btn.addEventListener("click", function () {
80
81          // 调用 postAxios 方法
82          postAxios('月海',16);
83
84      })
85
86  </script>
87
88  </html>
```

05、拦截器

点击

```
元素 控制台 源代码 网络 性能 » + 3
top ▼ 筛选器 默认级别 ▼ 3
响应拦截器 2，成功                                          05、拦截器.html:92
请求拦截器 2，成功                                          05、拦截器.html:72
请求拦截器 1，成功                                          05、拦截器.html:60
在请求拦截器 1 中设置参数完成                               05、拦截器.html:63
响应拦截器 1，成功                                          05、拦截器.html:81
返回的数据为：Hello Spring Boot 2 POST；姓名：月海，年龄：16   05、拦截器.html:83
                                                            05、拦截器.html:107
▼ {data: 'Hello Spring Boot 2 POST；姓名：月海，年龄：16', status: 200, statusText:
  '', headers: AxiosHeaders, config: {…}, …}
  ▶ config: {transitional: {…}, transformRequest: Array(1), transformResponse: Array
    data: "Hello Spring Boot 2 POST；姓名：月海，年龄：16"
  ▶ headers: AxiosHeaders {content-length: '56', content-type: 'application/json', S
  ▼ request: XMLHttpRequest
    ▶ onabort: ƒ handleAbort()
    ▶ onerror: ƒ handleError()
      onload: null
    ▶ onloadend: ƒ onloadend()
      onloadstart: null
      onprogress: null
      onreadystatechange: null
    ▶ ontimeout: ƒ handleTimeout()
      readyState: 4
      response: "Hello Spring Boot 2 POST；姓名：月海，年龄：16"
      responseText: "Hello Spring Boot 2 POST；姓名：月海，年龄：16"
      responseType: ""
      responseURL: "http://localhost:8080/hello?a=100&b=200"
      responseXML: null
      status: 200
      statusText: ""
      timeout: 0
    ▶ upload: XMLHttpRequestUpload {onloadstart: null, onprogress: null, onabort: nu
      withCredentials: false
    ▶ [[Prototype]]: XMLHttpRequest
    status: 200
    statusText: ""
  ▶ [[Prototype]]: Object
>
```

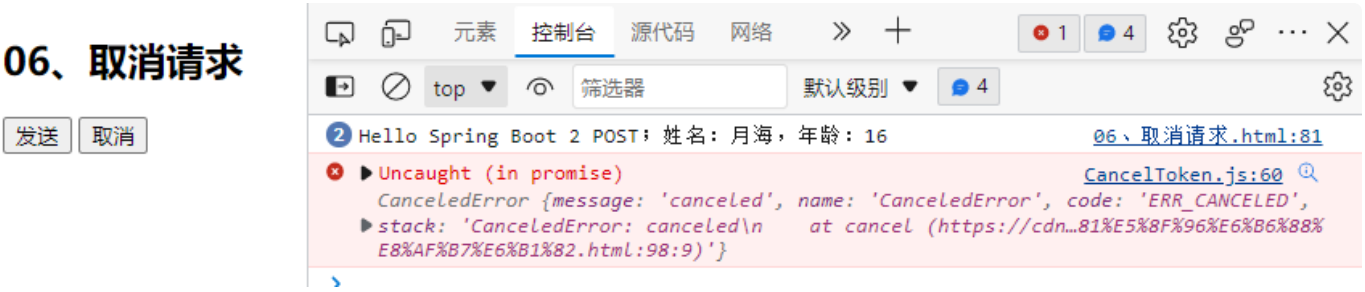
请求拦截器中配置的参数

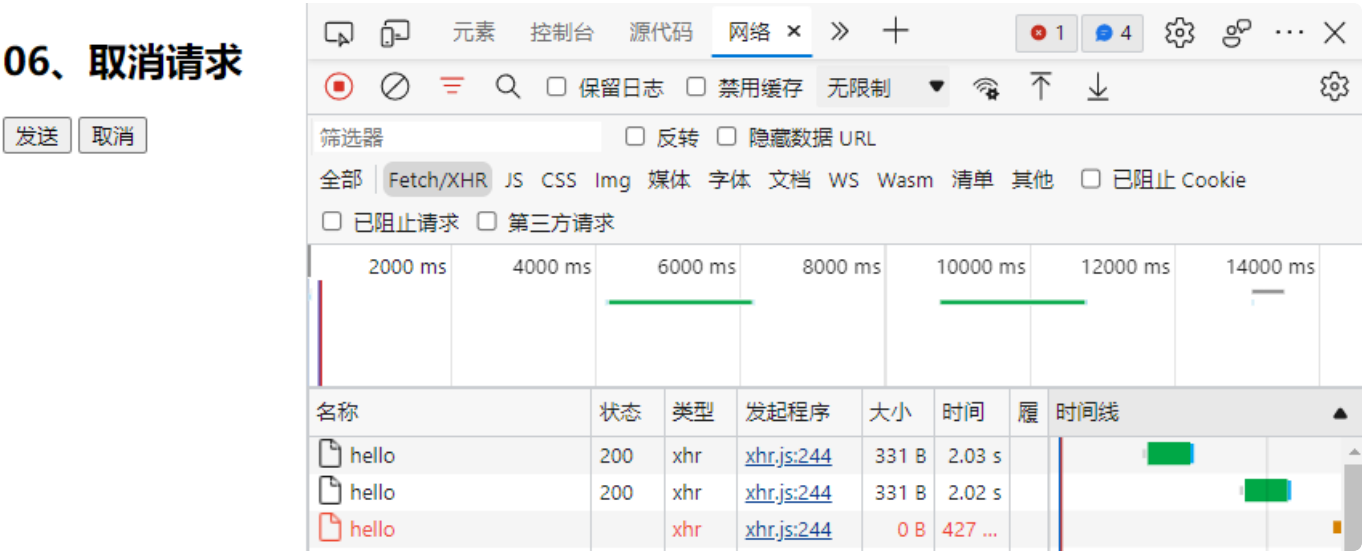
8、取消请求：`axios.CancelToken()`

HTML | 复制代码

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- 引入 axios -->
    <script src="https://cdn.bootcdn.net/ajax/libs/axios/1.1.3/axios.js">
</script>
    <title>Document</title>
</head>

<body>
    <h2>06、取消请求</h2>
    <button id="btn">发送</button>
    <button id="btn2">取消</button>
</body>

<script>

    // 进行 axios 默认配置
    // 配置默认请求类型
    axios.defaults.method = 'POST';
    // 配置基础 url
    axios.defaults.baseURL = 'http://localhost:8080';

    // 取消请求第二步：声明一个全局变量（函数）
    let cancel = null;

    // 定义使用 axios 发送 post 请求的函数
    function postAxios(name, age) {
        axios({
            url: '/hello',
            data: {
                name: name,
                age: age
            },
            // 取消请求第一步：添加配置对象属性 cancelToken
            cancelToken: new axios.CancelToken( c => {
                cancel = c;
            } )
        }).then(response =>{
            console.log(response.data);
        })
```

```
    }

    // 获取元素对象
    const btn = document.querySelector("#btn")
    // 绑定单击事件
    btn.addEventListener("click", function () {
        // 调用 postAxios 方法，发送请求
        postAxios('月海',16);
    })

    // 获取元素对象
    const btn2 = document.querySelector("#btn2")
    // 绑定单击事件
    btn2.addEventListener("click", function () {
        // 调用 cancel 方法，取消请求
        cancel();
    })

</script>

</html>
```





- 一个按钮实现 `发送/取消`

- 当有请求进行时，点击按钮取消请求
- 没有请求进行时，点击按钮发送请求

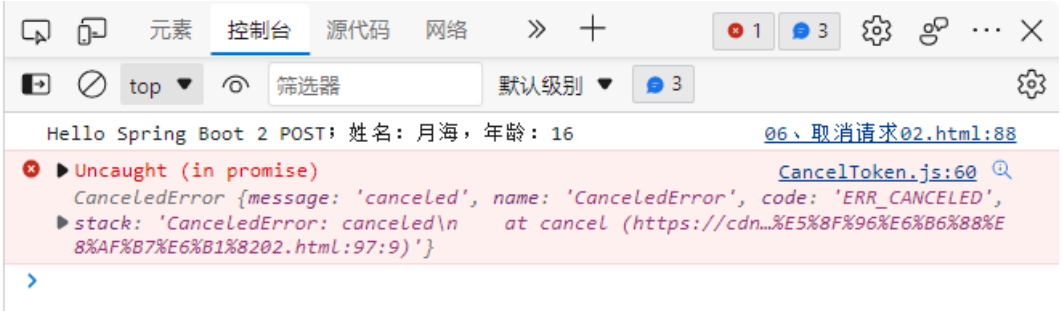
HTML | 复制代码

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- 引入 axios -->
    <script src="https://cdn.bootcdn.net/ajax/libs/axios/1.1.3/axios.js">
</script>
    <title>Document</title>
</head>

<body>
    <h2>06、取消请求02</h2>
    <button id="btn">发送/取消</button>
</body>

<script>

    // 进行 axios 默认配置
    // 配置默认请求类型
    axios.defaults.method = 'POST';
    // 配置基础 url
    axios.defaults.baseURL = 'http://localhost:8080';

    // 取消请求第二步：声明一个全局变量（函数）
    let cancel = null;

    // 定义使用 axios 发送 post 请求的函数
    function postAxios(name, age) {
        // 判断 cancel 的值是否为 null；为 null 表示没有正在进行的请求；不为 null
表示有请求正在进行
        if( cancel != null ){
            // cancel 不为 null，则使它变为 null，即取消请求
            cancel();
            return;
        }
        axios({
            url: '/hello',
            data: {
                name: name,
                age: age
            },
            // 取消请求第一步：添加配置对象属性 cancelToken
```

```
44                 cancelToken: new axios.CancelToken( c => {
45                     cancel = c;
46                 } )
47             }).then(response =>{
48                 // 请求完毕后，将 cancel 的值重新赋值为 null
49                 cancel = null;
50                 console.log(response.data);
51             })
52         }
53
54         // 获取元素对象
55         const btn = document.querySelector("#btn")
56         // 绑定单击事件
57         btn.addEventListener("click", function () {
58             // 调用 postAxios 方法，发送请求
59             postAxios('月海',16);
60         })
61
62     </script>
63
64     </html>
```

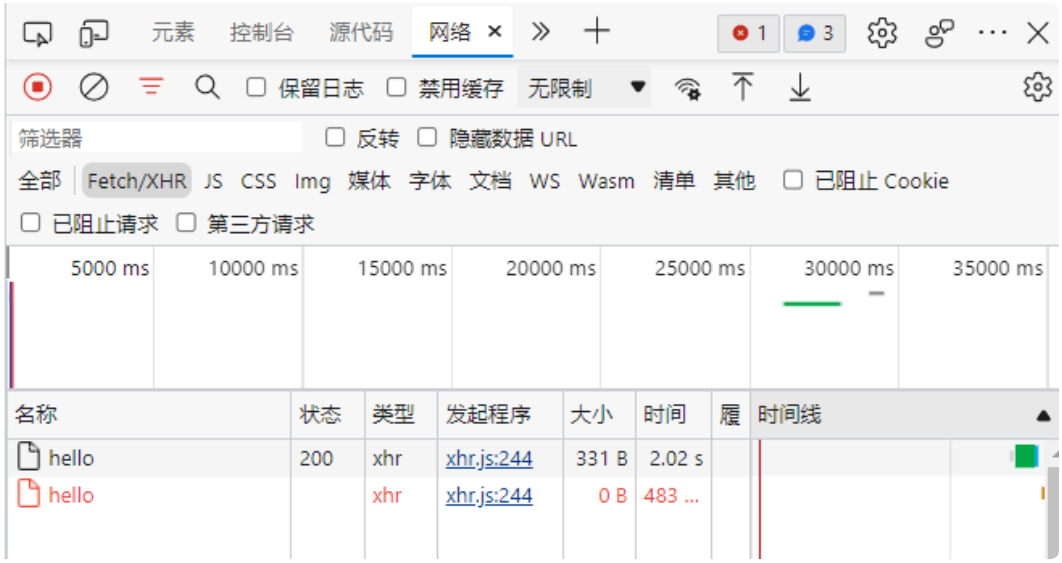
06、取消请求02

发送/取消



06、取消请求02

发送/取消



五、

六、

七、

1、

2、

3、

4、

5、

6、

7、

8、

9、

---

①、

②、

③、

④、

⑤、

⑥、

⑦、

⑧、

⑨、

⑩、

Ⅰ、

Ⅱ、

Ⅲ、

Ⅳ、

Ⅴ、

Ⅵ、

Ⅶ、

Ⅷ、

IX、

X、